ACN PROJECT · TEAM 7

Frame-aware multipath QUIC with a learned bandwidth predictor

Streaming live lectures over two mobile connections without the video freezing

On real cellular networks, standard schedulers deliver **16.4%–18.2%** of video frames too late. Ours: **10.6%–13.5%**.

Kabir Bajaj and Team 7 · 29 October 2026

Slide 1. A phone with two SIM cards has two internet connections. We send a live lecture over both at once, and we built a scheduler, helped by a small neural network, that decides which connection each video frame takes so it arrives in time. The box is the headline result.

The setting

- Students watch live lectures on a **phone with mobile data**
- Many phones have **two SIM cards**: two connections, each slow and unsteady (a few Mbit/s, 50–150 ms delay)
- **Multipath QUIC** uses both at once; a **scheduler** picks the path for every packet



I-frame: full picture, ~55 KB, every 2 s P-frame: only the changes, ~8 KB, 24 per second

Frame i, produced at time s_i, must arrive completely by $s_i + D$, with playback delay D **= 300 ms**. If not, it is a **deadline miss** and the video freezes.

Slide 2. QUIC is the modern transport protocol behind YouTube and Google; multipath QUIC lets one connection use two networks. The scheduler is the part that decides which network each packet goes on. Video is a sequence of frames: a big full picture (I-frame) every 2 seconds and small difference frames (P-frames) in between. The player shows each frame D = 300 ms after it was made; a frame that is late or incomplete causes a freeze. The big I-frames are the risky ones.

The problem: schedulers see bytes, not frames

- **MinRTT** (TQUIC's default): next packet on the lowest-delay path with room
- **Round-robin**: alternate between paths
- **ECF** (earliest completion first): wait for the fast path if that finishes sooner
- **Redundant**: every packet on both paths (half the capacity spent on copies)
- None knows where a frame starts or ends, or when it is due: a **split frame waits for its slower half**, and **small frames get stuck behind big ones**

EXAMPLE: ONE I-FRAME ON A SLOW PATH

$$\frac{45\ \text{KB} \times 8}{1.5\ \text{Mbit/s}} + 30\ \text{ms} = \mathbf{270\ ms}$$

That is 270 of the 300 ms budget, before counting frames already queued.

Research questions: (1) how often do standard schedulers miss deadlines? (2) can a scheduler that knows each frame's size and deadline, with a learned speed prediction, do better? (3) which part of the design matters?

Slide 3. MinRTT sends on the lowest-delay path that has room; round-robin alternates; ECF waits for the fast path if that would finish sooner; redundant sends everything twice. None of them knows about frames. The example: 45 kilobytes is 360 kilobits; at 1.5 megabits per second that is 240 ms of sending, plus 30 ms of travel. On slow links there is no room for a bad choice. Question 3 is answered with ablations: versions of our scheduler with one part removed.

Our scheduler: predict when each frame will arrive

$$T_p = \frac{Q_p + S}{B_p} + d_p$$

T_p arrival time on path p · S frame size · Q_p bytes waiting ahead of it · B_p path speed · d_p = $\text{sRTT}_p/2$ one-way delay

RULE 1: WHOLE FRAME IF A PATH MAKES IT

$$p^* = \operatorname{argmin}_p T_p$$
$$\text{use } p^* \text{ if } T_{p^*} \le h \cdot (\text{deadline} - \text{now})$$

Then all S bytes go on p^*. Safety margin h = 0.85 for the hand-tuned version, h = 1 for the learned one (its prediction is already cautious).

RULE 2: OTHERWISE SPLIT, BOTH PARTS FINISH TOGETHER

$$T^* = \frac{S + Q_A + Q_B + B_A d_A + B_B d_B}{B_A + B_B}$$
$$s_p = B_p(T^* - d_p) - Q_p$$

$s_A + s_B = S$, and both parts arrive at the same time T^*.

Q_p = bytes of earlier frames planned for p + $\max(0, \text{in-flight}_p - B_p \cdot \text{minRTT}_p)$. Each path sends only its own frames, so small frames overtake big ones. A frame already past its deadline may use any path.

Slide 4. The formula: to know when a frame arrives on a path, add up everything waiting ahead of it plus the frame itself, divide by the path's speed, and add the one-way delay. Rule 1: pick the path with the earliest arrival; if it makes the deadline, the whole frame goes there. The hand-tuned version keeps a 15% safety margin. Rule 2, mostly for big I-frames: split the frame so both parts land at the same time. Setting the two arrival times equal and the two parts summing to S gives T-star and each part's size. Q_p also counts data stuck in the network's queue: bytes in flight beyond what the path holds at its minimum RTT. Everything depends on B_p, the speed, which is what the neural network predicts.

The ML part: predicting the speed B_p

- If B_p is **too high**, rule 1 trusts a path that will be late. BBR's own estimate is too high **38.2%** of the time on our test recordings.
- **Input** x: 44 numbers per path from the last second (delivery rate, send rate, queueing delay, loss, cwnd, BBR's estimate, ...)
- **Target**: $y = \ln(1 + C/1000)$, C = true average speed over the next 0.5 s (bytes/s)
- **Data**: 6 recorded cellular networks for training, 2 others for testing

WHAT THE SCHEDULER USES

$$B_p = 1000\,(e^{\hat{y}_{0.1}} - 1) \;\Rightarrow\; P(C < B_p) \approx 10\%$$

SMALL NEURAL NETWORK (MLP): 44 → 32 → 32 → 4

$$h_1 = \mathrm{ReLU}(W_1 x + b_1)$$
$$h_2 = \mathrm{ReLU}(W_2 h_1 + b_2)$$
$$\hat{y} = W_3 h_2 + b_3$$

Four outputs $\hat{y}_\tau$: the 10th, 20th, 30th and 50th percentile (τ = 0.1, 0.2, 0.3, 0.5).

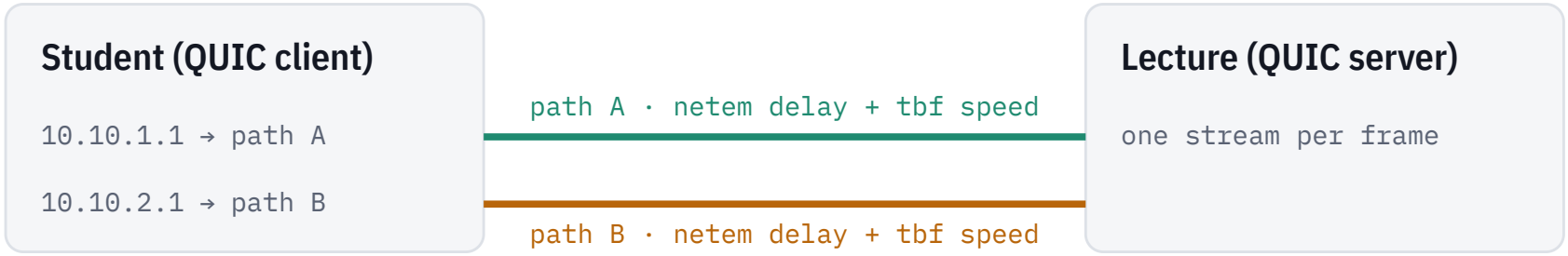
PINBALL LOSS

$$L_\tau(y, \hat{y}) = \max(\tau(y - \hat{y}), (\tau - 1)(y - \hat{y}))$$

For τ = 0.1, too high costs 0.9, too low 0.1: **9× penalty for overestimates**.

Slide 5. Motivation first: the formula is only as good as the speed estimate, and a high estimate is the dangerous error. The model reads the last second of each path and predicts the speed for the next half second, in a log scale so that slow and fast links are treated alike. It is a two-layer network; ReLU just sets negative values to zero. It outputs four percentiles instead of one guess. The pinball loss is the standard loss for percentiles: for the 10th percentile, an overestimate is penalised nine times more than an underestimate, so the model learns a value the real speed stays above 90% of the time. That cautious value is B_p. The weights are compiled into TQUIC's Rust code, and Rust and Python agree to 0.002%.

How we test: real QUIC code on an emulated network



- One laptop: Linux **network namespaces** joined by two virtual links (**veth**)
- Linux **tc**: netem adds delay, tbf sets speed with a 100 ms queue; every 100 ms the speed is replayed from **real cellular recordings**
- A real 60 s lecture (1,441 frames, 1.57 Mbit/s); 8 schedulers × 5 scenarios × 5 runs = **200 runs**, same network for the same run number

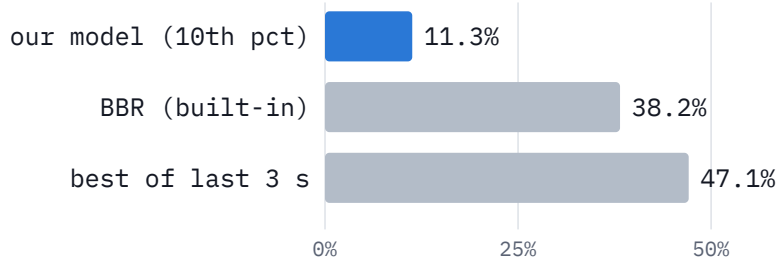
SCENARIO	PATH A	PATH B
symmetric	4 Mbit/s, 40 ms	4 Mbit/s, 40 ms
rtt_asym	4 Mbit/s, 30 ms	4 Mbit/s, 150 ms
low_bw	1.5 Mbit/s, 60 ms	1 Mbit/s, 120 ms
trace	AT&T LTE	T-Mobile 3G
outage	4 Mbit/s	off 2 s / 30 s

Slide 6. The emulator is two isolated network stacks on one laptop, connected by two virtual cables, each shaped with Linux traffic control. A script replays recorded cellular speeds every 100 ms. The real TQUIC code runs on top, so behaviour and bugs are real. The trace scenario uses the two recordings the model never saw; it is the realistic one. Runs with the same number see exactly the same network, so schedulers are compared fairly, run by run.

Result 1: the model predicts speed better

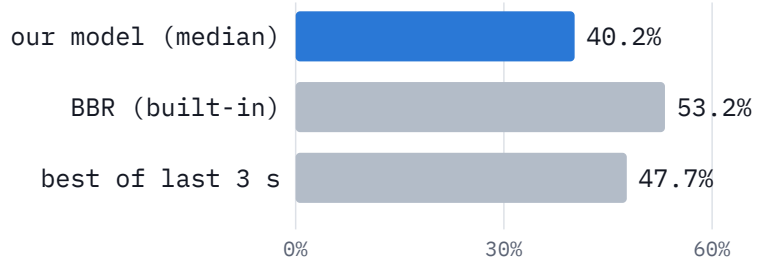
How often the guess is too high

$$\mathrm{P}(\hat{B} > C)$$



Typical error

$$\text{median } |\hat{B} - C| \,/\, C$$

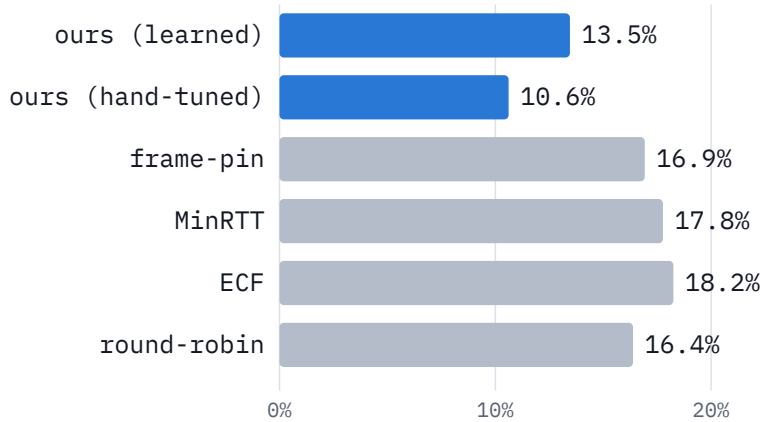


$\hat{B}$ = estimate, C = true speed over the next 0.5 s, at 31,912 moments on the 2 recordings the model never saw. Blue = our model. **Shorter bars are better.**

The 10th percentile is too high **11.3%** of the time, close to the 10% it promises (**calibrated**), against **38.2%** for BBR.

Slide 7. Left chart: how often each estimate was above the true speed, the dangerous mistake. Our cautious estimate is too high about 11% of the time, which is what a 10th percentile should do; BBR is too high 38% of the time, and the best-of-last-3-seconds rule (what our hand-tuned version starts from) 47%. Right chart: the typical relative error. Our median prediction is closer to the truth than both rules.

Result 2: fewer late frames on real cellular networks



Trace scenario, median of 5 runs. Blue = ours. **Shorter is better.** Redundant 52.8% and single path 72.9% are off the scale.

$$\text{miss rate} = \frac{1}{N} \Sigma_i \, 1[c_i > s_i + D]$$

c_i = when frame i is complete, N = 1,441 frames, D = 300 ms

- Late frames: **10.6%–13.5%** with ours vs **16.4%–18.2%** standard
- Run by run, learned beats MinRTT and ECF in 3 and 3 of 5 runs; freezes less in 4 and 4 of 5 (1.3 s vs 1.6–1.8 s per minute)
- **Ablation**: frame-pin keeps frames whole but has no deadline check or split: 16.9%, no better than standard

Slide 8. The chart shows, for each scheduler, the share of the lecture's frames that arrived after their deadline on the real recordings. Both of our versions beat every standard scheduler, and the order is the same with 150 ms and 500 ms deadlines. Freezing time models a player that waits for late frames: it shows frame i at max(previous + frame interval, completion time), and every extra wait counts. The frame-pin ablation shows the deadline check and the split are what help, not just keeping frames whole. On fast links every scheduler is fine; on the slow constant links (3.3%–4.9% missed) the big I-frames barely fit a 300 ms budget for anyone.

Conclusions

WHAT WE FOUND

- Planning frames against deadlines cuts late frames from **16.4%–18.2%** to **10.6%–13.5%**
- Our ML model is the **most accurate** speed estimate
- But it is too cautious: the hand-tuned version is still a bit better (10.6% vs 13.5%; learned better in 2 of 5 runs)

LIMITATIONS

- Emulated network, not a live one
- Recordings from US networks
- One 60 s lecture, 5 runs each

NEXT STEPS

- Choose τ from the predicted spread: **less caution** on steady links
- Retrain on the learned scheduler's own traffic
- More recordings, longer lectures

Thank you · Questions?

Slide 9. Say the honest result clearly: deadline-aware scheduling works, and the model predicts speed better than anything else, but it does not yet beat the hand-tuned version because the 10th percentile is cautious even when the link is steady. Choosing the percentile from how spread out the four predictions are is the first next step. Along the way we also found and fixed a stall bug in TQUIC itself. Then invite questions.